Restricciones de integridad y transacciones

Clase 21

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Orden de los joins (anterior)

Materialización (anterior)

Restricciones de intergridad

Transacciones

ACID

Outline

Orden de los joins (anterior)

Materialización (anterior)

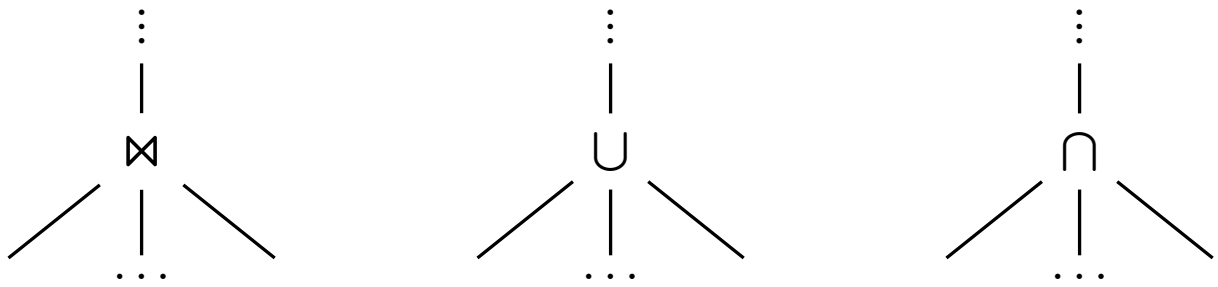
Restricciones de intergridad

Transacciones

ACID

Orden de operadores conmutativos y asociativos

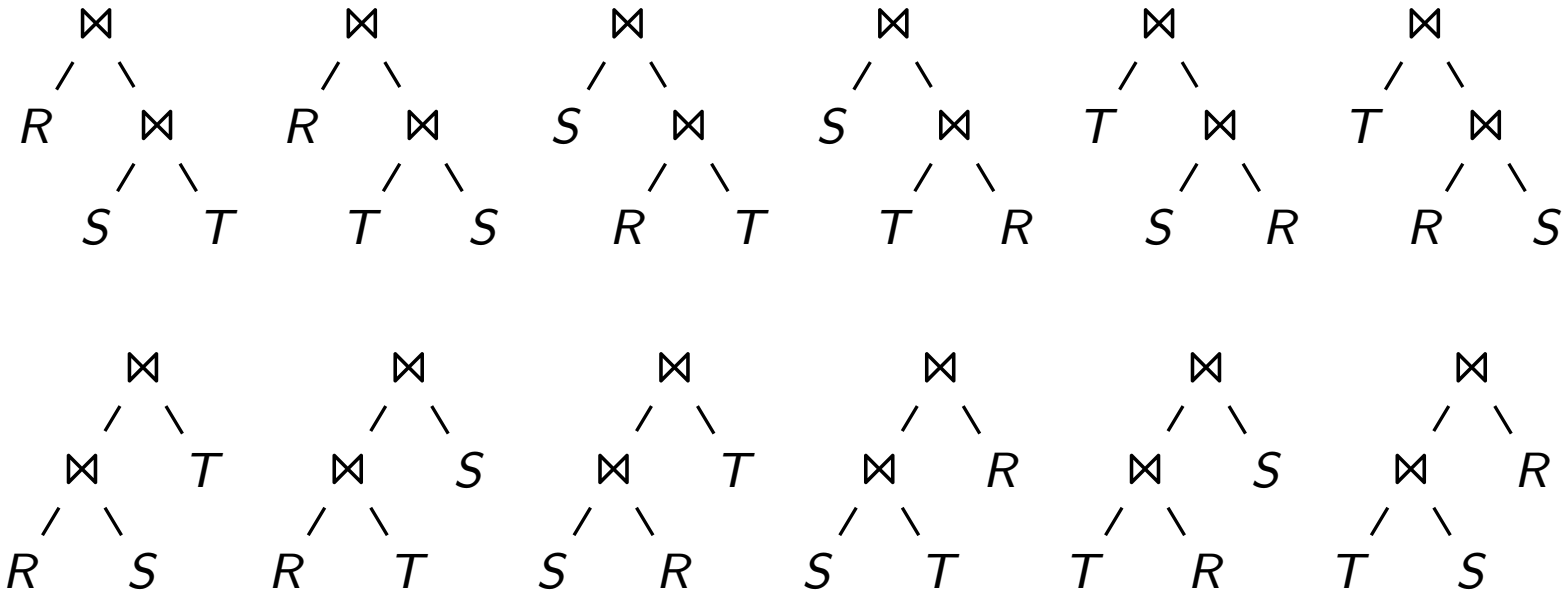
Dado un plan de lógico con nodos de la forma:



¿qué **orden** de los operadores usamos en el plan físico?

Orden de joins

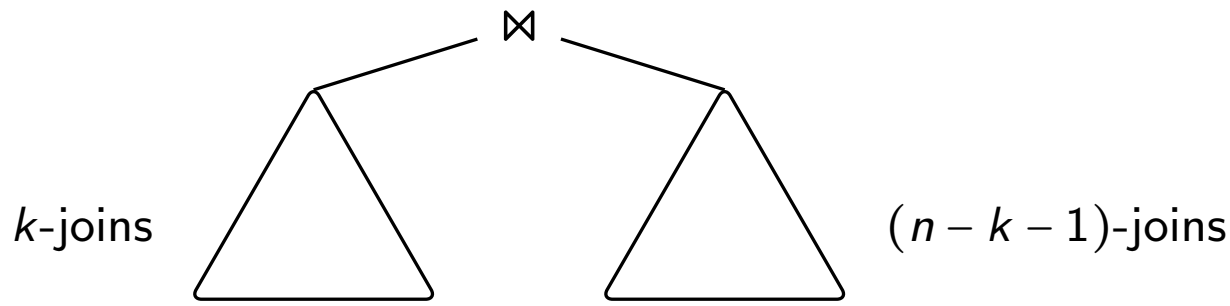
Para $R \bowtie S \bowtie T$, tenemos todos estos **planes** posibles:



¿cuál de todos escogemos?

¿cuántos planes de joins podemos escoger?

Para n-joins de la forma $R_1 \bowtie R_2 \bowtie \cdots \bowtie R_{n+1}$:



El número de posibles árboles de join para n-joins:

$$C_n = \sum_{k=0}^{n-1} C_k \cdot C_{n-k-1}$$

(conocidos como "Catalan numbers")

¿cuántos planes de joins podemos escoger?

Resolviendo esta ecuación recursiva:

$$C_n = \frac{(2n)!}{(n+1)!n!}$$

Multiplicando por todas las permutaciones $n!$ de las n relaciones:

$$\#\text{-planes de joins} = \frac{(2n)!}{(n+1)!}$$

Una **cantidad exponencial** de posibles planes!

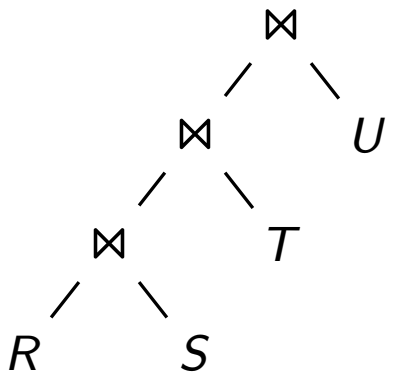
¿y ahora quién podrá defendernos?

Tres opciones para enumerar planes de joins:

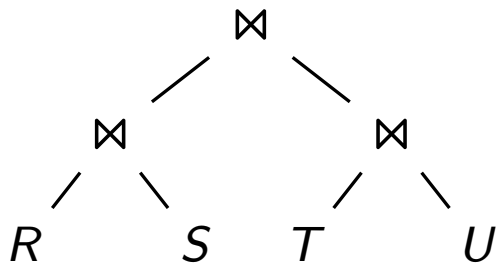
1. Considerarlos todos.
2. Considerar un subconjunto de los planes.
3. Usar una heurística.

2. Considerar un subconjunto de los planes

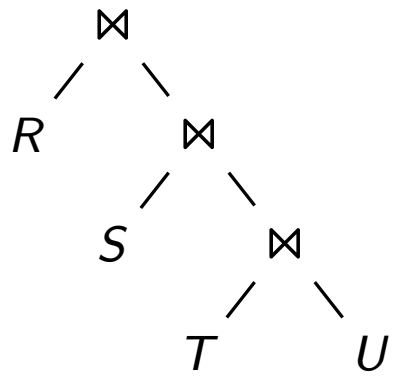
Distintos tipos de planes:



Left-deep plan



Bushy plan



Right-deep plan

¿qué **ventaja** tiene cada plan?

2. Considerar un subconjunto de los planes

Considerar solo **left-deep** plans.

Optimizador de **Selinger**

Ventajas

- Un subconjunto menor de planes a considerar.
- Buena interacción con los operadores físicos de join.

3. Usar una heurística

Heuristica **greedy** para: $R_1 \bowtie R_2 \bowtie \ldots \bowtie R_n$

- Mantener un conjunto de planes $\mathcal{P}$.
- Iniciar el conjunto de planes $\mathcal{P}$ con todas las relaciones:

$$\mathcal{P} = \{R_1, \ldots, R_n\}$$

- En cada iteración, escoger dos planes $P_1, P_2 \in \mathcal{P}$ tal que:

$$\text{cost}(P_1 \bowtie P_2) = \min_{P, P' \in \mathcal{P}} \text{cost}(P \bowtie P')$$

y reemplazar $\mathcal{P} := (\mathcal{P} - \{P_1, P_2\}) \cup \{P_1 \bowtie P_2\}$.

¿cuál es el **tiempo** de esta heurística?

Outline

Orden de los joins (anterior)

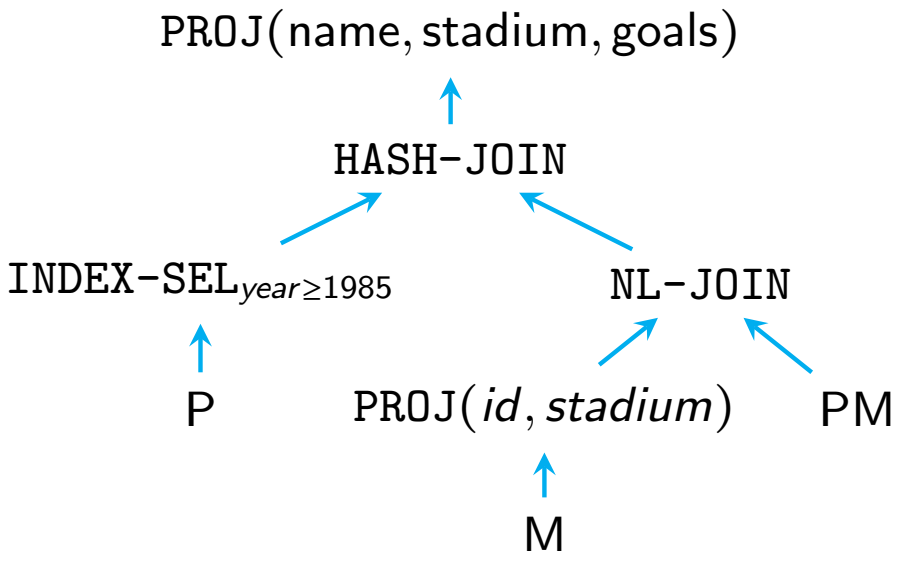
Materialización (anterior)

Restricciones de intergridad

Transacciones

ACID

Pipeline



- Ejecución **en serie** del plan físico.
- Cada operador genera **una tupla a la vez**.

Cada operador retorna **uno a uno** sus resultados al operador padre

Bloqueo de operadores

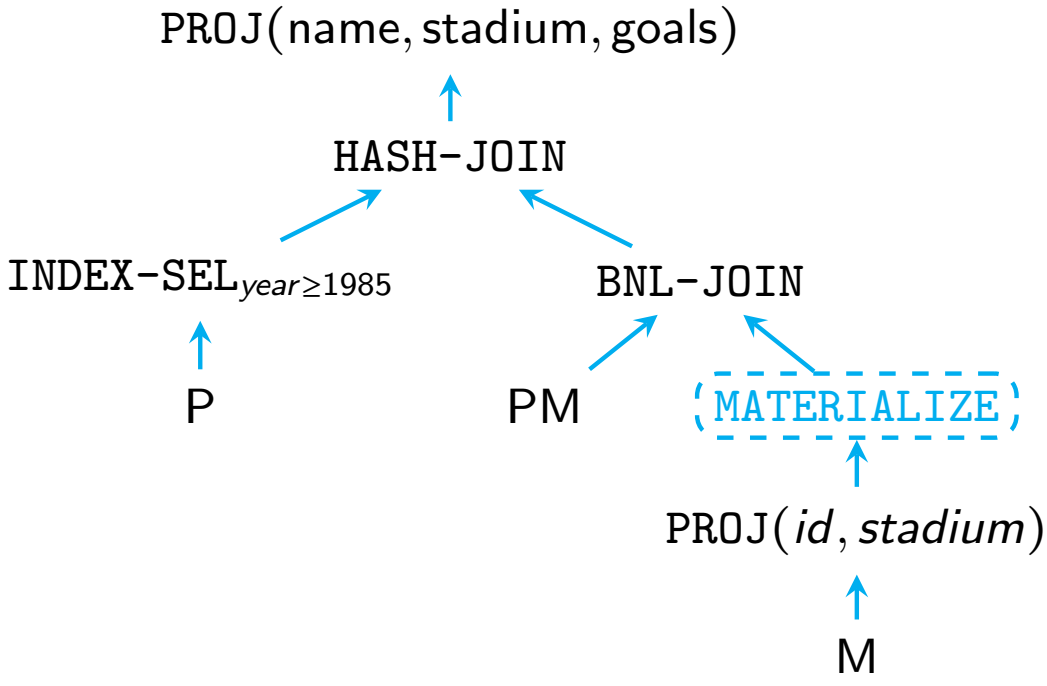
Algunos operadores deben esperar a terminar todo el proceso:

- external-sorting.
- joins.
- group-by y eliminación de duplicados.

Estos operadores usualmente requieren:

materialización (alternativa a Pipeline).

Materialización



- Almacena el resultado de la subconsulta en disco.
- Solo necesario si la subconsulta es requerida reiteradas veces.

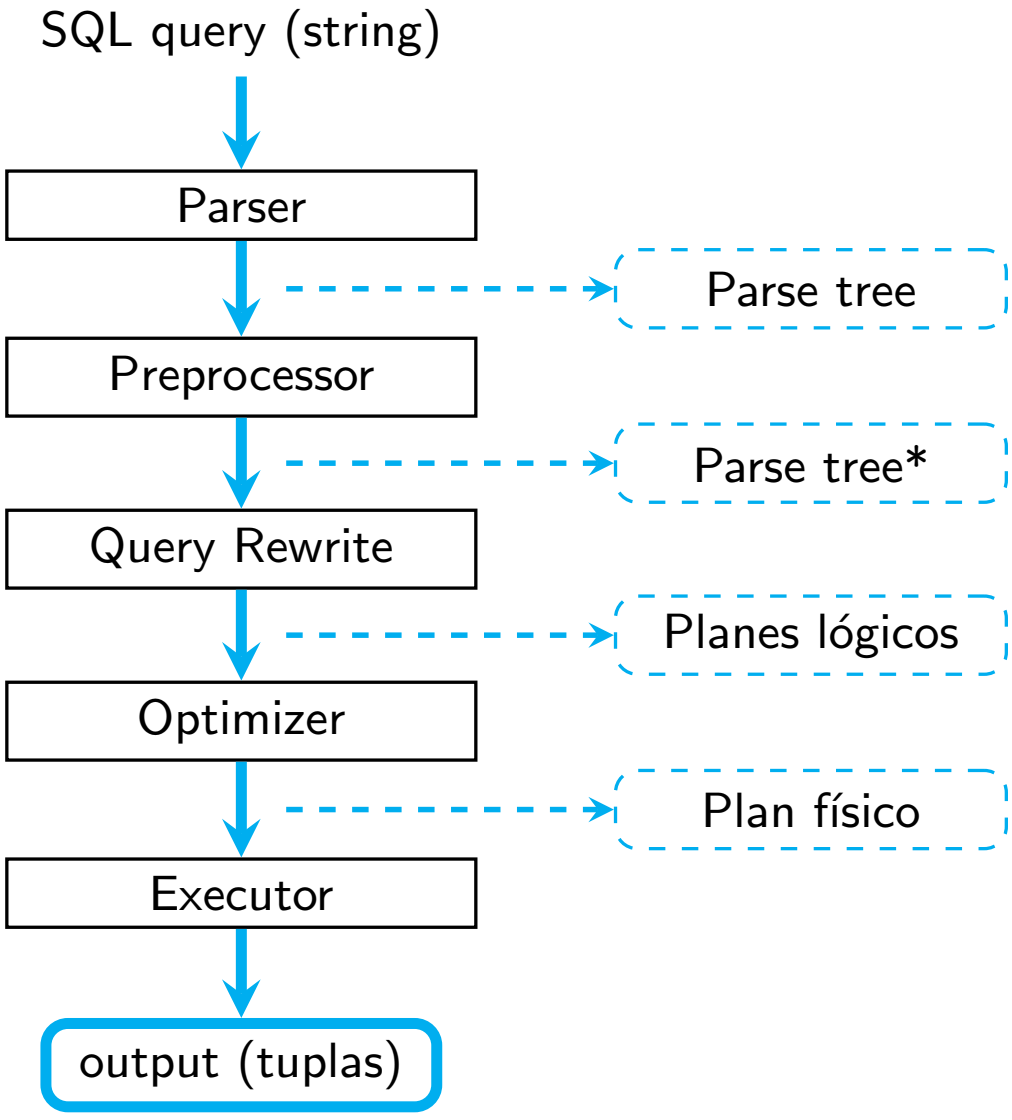
Pipeline vs Materialización

Siempre se prefiere pipeline sobre materialización

Usar materialización si:

- Subconsulta es llamada reiteradas veces.
- Costo de almacenamiento es menor a recalcular la consulta.

Finalmente, ejecutamos nuestro plan físico de consultas...



Outline

Orden de los joins (anterior)

Materialización (anterior)

Restricciones de intergridad

Transacciones

ACID

Restricciones de integridad

Definición

Una **restricción de integridad** es una regla (en SQL)
que una base de datos debe cumplir en todo momento
para asegurar la **correctitud**, **consistencia** y **valides** de los datos.

Tipos de restricciones de integridad en SQL

1. Data types
2. `PRIMARY KEY` y `UNIQUE`
3. `FOREIGN KEY`
4. `NOT NULL`
5. `CHECK` en atributos
6. `CHECK` en tuplas
7. `ASSERTION`

Tipos de restricciones de integridad

1. Data types

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	500	2.000
0002	Bob	0	1.000

Movements

time	aid	amount	comment
10:40	0001	−500	Cajero
12:15	0001	1000	Deposito

```
CREATE TABLE Accounts (
  id CHAR(4),
  userid VARCHAR(20),
  balance INT,
  credit INT);
```

```
CREATE TABLE Movements (
  time TIMESTAMP,
  aid CHAR(4),
  amount INT,
  comment VARCHAR(50));
```

- `INSERT INTO Accounts (id, userid, balance, credit) VALUES('0002', 'Bob', 1200, 1000);` ✓
- `INSERT INTO Accounts (id, userid, balance, credit) VALUES(2, 'Bob', 1200, 1000);` ✗

Tipos de restricciones de integridad

1. Data types
2. `PRIMARY KEY` y `UNIQUE`

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	500	2.000
0002	Bob	0	1.000

Movements

time	aid	amount	comment
10:40	0001	−500	Cajero
12:15	0001	1000	Deposito

```
CREATE TABLE Accounts (
   id CHAR(4) PRIMARY KEY,
   userid VARCHAR(20) UNIQUE,
   balance INT,
   credit INT);
```

```
CREATE TABLE Movements (
   time TIMESTAMP,
   aid CHAR(4),
   amount INT,
   comment VARCHAR(50),
   PRIMARY KEY (time, aid));
```

- `UPDATE Accounts SET userid = 'Bob' WHERE userid = 'Alice';` ✗

Tipos de restricciones de integridad

3. FOREIGN KEY

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	500	2.000
0002	Bob	0	1.000

Movements

time	aid	amount	comment
10:40	0001	−500	Cajero
12:15	0001	1000	Deposito

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20) UNIQUE,
  balance INT,
  credit INT);
```

```
CREATE TABLE Movements (
  time TIMESTAMP,
  aid CHAR(4),
  amount INT,
  comment VARCHAR(50),
  PRIMARY KEY (time, aid),
  FOREIGN KEY (aid)
    REFERENCES Accounts(id));
```

- `INSERT INTO Movements (time, aid, amount, comment)`
  `VALUES('12:20', '0003', -500, 'Tarjeta');`



Tipos de restricciones de integridad

4. NOT NULL

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	500	2.000
0002	Bob	0	1.000

Movements

time	aid	amount	comment
10:40	0001	−500	Cajero
12:15	0001	1000	Deposito

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20) UNIQUE,
  balance INT NOT NULL,
  credit INT NOT NULL);
```

```
CREATE TABLE Movements (
  time TIMESTAMP,
  aid CHAR(4),
  amount INT NOT NULL,
  comment VARCHAR(50),
  PRIMARY KEY (time, aid),
  FOREIGN KEY (aid)
    REFERENCES Accounts(id));
```

- `INSERT INTO Accounts (id, userid, balance, credit)`
  `VALUES('0003', 'Carl', NULL, 0);`



Tipos de restricciones de integridad

5. CHECK en atributos

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	500	2.000
0002	Bob	0	1.000

Movements

time	aid	amount	comment
10:40	0001	−500	Cajero
12:15	0001	1000	Deposito

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20) UNIQUE,
  balance INT NOT NULL,
  credit INT NOT NULL
    CHECK (credit >= 0));
```

```
CREATE TABLE Movements (
  time TIMESTAMP,
  aid CHAR(4),
  amount INT NOT NULL,
  comment VARCHAR(50),
  PRIMARY KEY (time, aid),
  FOREIGN KEY (aid)
    REFERENCES Accounts(id));
```

- `INSERT INTO Accounts (id, userid, balance, credit)`
  `VALUES('0003', 'Carl', 1000, -100);`



Tipos de restricciones de integridad

6. CHECK en tuplas

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	500	2.000
0002	Bob	0	1.000

Movements

time	aid	amount	comment
10:40	0001	−500	Cajero
12:15	0001	1000	Deposito

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20) UNIQUE,
  balance INT NOT NULL,
  credit INT NOT NULL
    CHECK (credit >= 0),
  CHECK (balance + credit >= 0));
```

```
CREATE TABLE Movements (
  time TIMESTAMP,
  aid CHAR(4),
  amount INT NOT NULL,
  comment VARCHAR(50),
  PRIMARY KEY (time, aid),
  FOREIGN KEY (aid)
    REFERENCES Accounts(id));
```

- `UPDATE Accounts SET balance = -1500 WHERE id = '0002';` ✗

Tipos de restricciones de integridad

6. CHECK en tuplas

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	500	2.000
0002	Bob	0	1.000

Movements

time	aid	amount	comment
10:40	0001	−500	Cajero
12:15	0001	1000	Deposito

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20) UNIQUE,
  balance INT NOT NULL,
  credit INT NOT NULL
    CHECK (credit >= 0),
  CHECK (balance + credit >= 0));
```

```
CREATE TABLE Movements (
  time TIMESTAMP,
  aid CHAR(4),
  amount INT NOT NULL,
  comment VARCHAR(50),
  PRIMARY KEY (time, aid),
  FOREIGN KEY (aid)
    REFERENCES Accounts(id));
```

¿cuál es la **diferencia** de CHECK en attributos versus tuplas?

Tipos de restricciones de integridad

7. ASSERTION

Ejemplo

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20) UNIQUE,
  balance INT NOT NULL,
  credit INT NOT NULL
    CHECK (credit >= 0),
  CHECK (balance + credit >= 0));
```

```
CREATE TABLE Movements (
  time TIMESTAMP,
  aid CHAR(4),
  amount INT NOT NULL,
  comment VARCHAR(50),
  PRIMARY KEY (time, aid),
  FOREIGN KEY (aid)
    REFERENCES Accounts(id));
```

```
CREATE ASSERTION balance_movements
CHECK ( NOT EXISTS (
     SELECT * FROM Accounts A
     WHERE A.balance <> ( SELECT SUM(M.amount)
                          FROM Movements M
                          WHERE M.aid = A.id) ) );
```

ASSERTION **no son soportadas** por todos los motores (ej. PostgreSQL)

Outline

Orden de los joins (anterior)

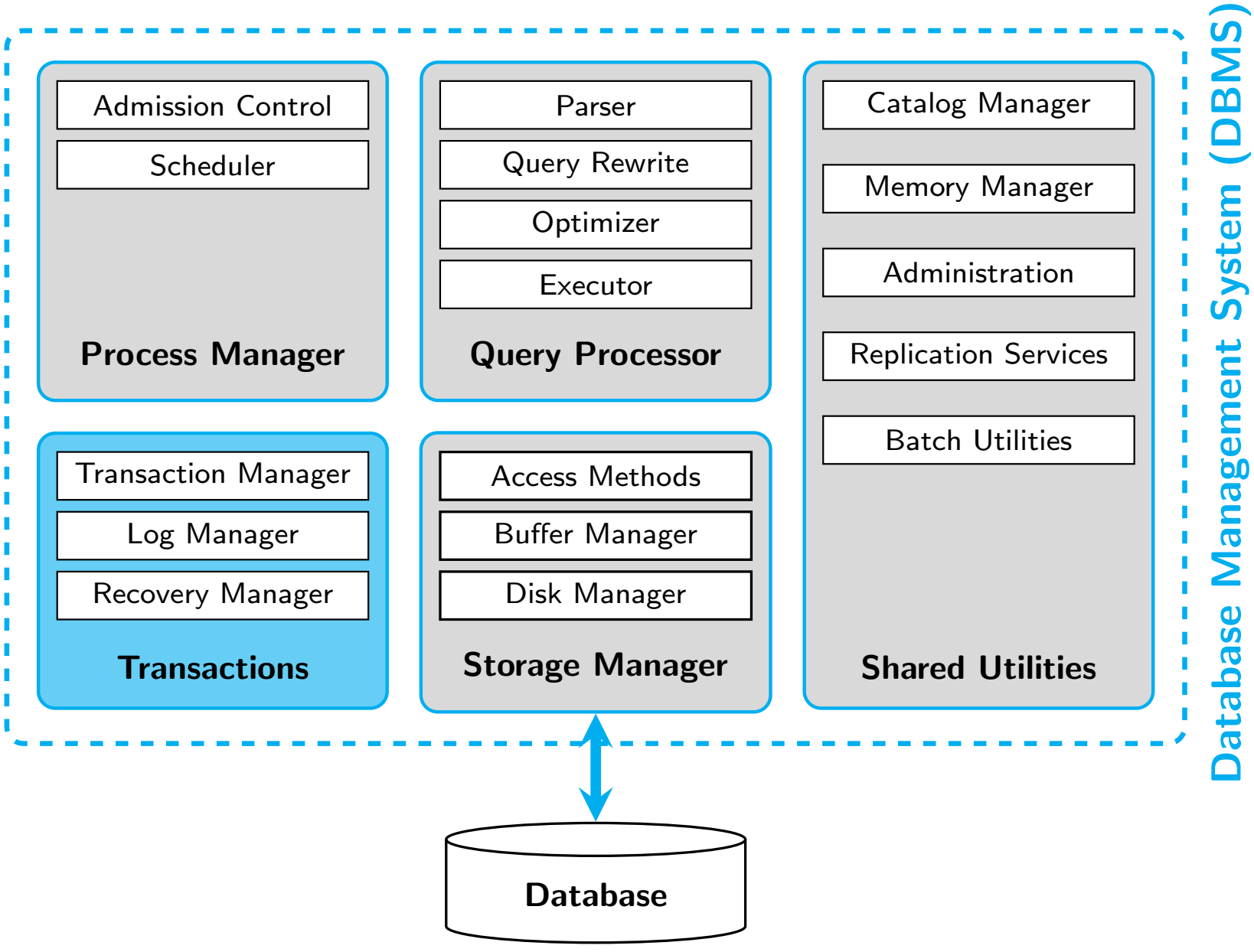
Materialización (anterior)

Restricciones de intergridad

Transacciones

ACID

Gestión de transacciones



Acceso a los datos

Hasta el momento hemos supuesto:

1. un solo proceso accede los datos.
2. cada consulta o actualización es **atómica**.
3. no hay **fallas** en la administración de los datos.

¿qué tan **razonables** son estas suposiciones?

Acceso a los datos

Ejemplo: traspaso de dinero entre cuentas

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

Movements

time	aid	amount	comment
10:40	0001	+1000	Cajero
12:15	0002	+200	Deposito
13:30	0002	+800	Deposito

```
UPDATE   Accounts
SET      balance = balance - 100
WHERE    id = '0001';
```

$$\begin{array}{l} \texttt{READ}(b_1, 100) \\ \texttt{WRITE}(b_1, x - 100) \end{array}$$

```
UPDATE   Accounts
SET      balance = balance + 100
WHERE    id = '0002';
```

$$\begin{array}{l} \texttt{READ}(b_2, 100) \\ \texttt{WRITE}(b_2, x + 100) \end{array}$$

Acceso a los datos

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2	
$\texttt{READ}(b_1, x)$		1000	1000	
$\texttt{WRITE}(b_1, x - 100)$		900		
$\texttt{READ}(b_2, x)$				
$\texttt{WRITE}(b_2, x + 100)$			1000	
	$\texttt{READ}(b_1, y)$			
	$\texttt{WRITE}(b_1, y - 100)$	800		
	$\texttt{READ}(b_2, y)$			
	$\texttt{WRITE}(b_2, y + 100)$	800	1200	✓

Acceso a los datos

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2	
`READ`(b_1, x)		1000	1000	
`WRITE`$(b_1, x - 100)$		900		
	`READ`(b_1, y)			
	`WRITE`$(b_1, y - 100)$	800		
	`READ`(b_2, y)			
	`WRITE`$(b_2, y + 100)$		1100	
`READ`(b_2, x)				
`WRITE`$(b_2, x + 100)$		800	1200	✓

Acceso a los datos

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2	
$\texttt{READ}(b_1, x)$		1000	1000	
$\texttt{WRITE}(b_1, x - 100)$		900		
$\texttt{READ}(b_2, x)$				
	$\texttt{READ}(b_1, y)$			
	$\texttt{WRITE}(b_1, y - 100)$	800		
	$\texttt{READ}(b_2, y)$			
	$\texttt{WRITE}(b_2, y + 100)$		1100	
$\texttt{WRITE}(b_2, x + 100)$		800	1100	✕

Acceso a los datos

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2	
`READ`(b_1, x)		1000	1000	
`WRITE`$(b_1, x - 100)$		900		
`ERROR`	`ERROR`	900	1000	✕

Necesitamos transacciones

Definición

Una **transacción** es una secuencia
de una o más operaciones que modifican o consultan la base de datos.

Ejemplos

- Transferencias de dinero entre cuentas de banco.
- Compra de un grupo de bienes.
- Registrarse para un curso.

Transacciones en SQL

```
START TRANSACTION;
    <una-o-mas-consultas-o-actualizaciones>
COMMIT;
```

Ejemplo

```
START TRANSACTION;

    UPDATE Accounts
    SET balance = balance - 100
    WHERE id = '0001';

    UPDATE Accounts
    SET balance = balance + 100
    WHERE id = '0002';

COMMIT;
```

Permite agrupar operaciones **en una sola transacción**

Transacciones en SQL

Importante

En ausencia de `START TRANSACTION ... COMMIT`
cada consulta es una transacción **por si sola**.

Ejemplo

```
UPDATE Accounts
SET balance = balance - 100
WHERE id = '0001';

UPDATE Accounts
SET balance = balance + 100
WHERE id = '0002';
```

≡

```
START TRANSACTION;
  UPDATE Accounts
  SET balance = balance - 100
  WHERE id = '0001';
COMMIT;
START TRANSACTION;
  UPDATE Accounts
  SET balance = balance + 100
  WHERE id = '0002';
COMMIT;
```

Transacciones en SQL: atomicidad

¿cuál es el resultado de esta transacción?

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

```
CREATE TABLE Accounts (
   ...
   CHECK (balance + credit >= 0));
```

```
START TRANSACTION;

      UPDATE Accounts
      SET balance = balance + 3000
      WHERE id = '0001';

      UPDATE Accounts
      SET balance = balance - 3000
      WHERE id = '0002';

COMMIT;
```

Permite asegurar que cada transacción es **atómica**

(se hace entera o no se hace)

Transacciones en SQL: ROLLBACK

```
START TRANSACTION;
    <una-o-mas-consultas-o-actualizaciones>
ROLLBACK;
```

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

Movements

time	aid	amount	comment
10:40	0001	+1000	Cajero
12:15	0002	+200	Deposito
13:30	0002	+800	Deposito

```
START TRANSACTION;

    UPDATE Accounts
    SET balance = balance + 100
    WHERE id = '0001';

ROLLBACK
```

ROLLBACK **deshace los cambios** desde el inicio de la transacción

Transacciones en SQL: SAVEPOINT

```
SAVEPOINT <nombre-savepoint>;
<una-o-mas-consultas-o-actualizaciones>
ROLLBACK TO SAVEPOINT <nombre-savepoint>;
```

Ejemplo

```
START TRANSACTION;
      UPDATE Accounts
      SET balance = balance - 100
      WHERE id = '0001';
      SAVEPOINT half-transaction;
      UPDATE Accounts
      SET balance = balance + 100
      WHERE id = '0002';
      ROLLBACK TO SAVEPOINT half-transaction;
COMMIT;
```

SAVEPOINT permite hacer ROLLBACK **hasta algún punto** de la transacción

Transacciones en SQL: restricciones de integridad

Por defecto, durante una transacción
las restricciones son verificadas después de cada operación.

¿cuál es el resultado de esta transacción?

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

Movements

time	aid	amount	comment
10:40	0001	+1000	Cajero
12:15	0002	+200	Deposito
13:30	0002	+800	Deposito

```
START TRANSACTION;
      INSERT INTO Accounts
      VALUES ('0001', 'Ana', 1000, 500);
      DELETE FROM Accounts
      WHERE id = '0001' AND userId = 'Alice';
COMMIT;
```

En algunos casos, es necesario **levantar las restricciones** momentáneamente

Transacciones en SQL: DEFERRABLE

Ejemplo

```
CREATE TABLE Accounts (
   id CHAR(4) PRIMARY KEY,
   userid VARCHAR(20),
   ...
   CONSTRAINT uniqueuserid UNIQUE (userid)
   DEFERRABLE INITIALLY IMMEDIATE);


START TRANSACTION;
      SET CONSTRAINTS uniqueuserid DEFERRED;
      INSERT INTO Accounts
      VALUES ('0001', 'Ana', 1000, 500);
      DELETE FROM Accounts
      WHERE id = '0001' AND userId = 'Alice';
COMMIT;
```

DEFFERABLE permite **diferir la restricción** hasta el COMMIT

Transacciones en SQL: DEFERRABLE

Ejemplo (siempre diferir)

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20),
  ...
  CONSTRAINT uniqueuserid UNIQUE (userid)
  DEFERRABLE INITIALLY DEFERRED);


START TRANSACTION;
      INSERT INTO Accounts
      VALUES ('0001', 'Ana', 1000, 500);
      DELETE FROM Accounts
      WHERE id = '0001' AND userId = 'Alice';
COMMIT;
```

`INITIALLY DEFERRED` no requiere usar `SET CONSTRAINTS ... DEFERRED`

Sobre transacciones

- Unos de los componentes principales de una base de datos.
- Fundamental para muchas aplicaciones.
- Uno de los premios **Turing Award** en BD:

Charles Bachman	1973	primeros cimientos para DBMS.
Edgar Codd	1981	por inventar el modelo relacional.
Jim Gray	1998	por inventar las **transacciones**.
Michael Stonebracker	2015	por desarrollar Ingres.

Outline

Orden de los joins (anterior)

Materialización (anterior)

Restricciones de intergridad

Transacciones

ACID

Garantías ACID

ACID = **A**tomicity
Consistency
Isolation
Durability

Todos los sistemas de bases de datos **deben garantizar ACID**

(pero no todos lo hacen)

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Ejemplo

```
START TRANSACTION;

      UPDATE Accounts
      SET balance = balance - 100
      WHERE id = '0001';

      UPDATE Accounts
      SET balance = balance + 100
      WHERE id = '0002';

COMMIT;
```

¿qué pasa si justo **falla el sistema** después del primer update?

Se ejecutan ambos updates o no se ejecuta ninguno.

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Consistency

Al terminar una transacción los datos deben ser
correctos y consistentes según las restricciones de integridad.

Garantías ACID

Consistency

Al terminar una transacción los datos deben ser correctos y consistentes según las restricciones de integridad.

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

```
CREATE TABLE Accounts (
   ...
   CHECK (balance + credit >= 0));
```

```
START TRANSACTION;
      UPDATE Accounts SET balance = balance + 3000
      WHERE id = '0001';
      UPDATE Accounts SET balance = balance - 3000
      WHERE id = '0002';
COMMIT;
```

Si el estado final no es consistente, la transacción fallará.

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Consistency

Al terminar una transacción los datos deben ser
correctos y consistentes según las restricciones de integridad.

Isolation

Cada transacción se ejecuta sin ser interferida por otras transacciones.

Garantías ACID

Isolation

Cada transacción se ejecuta sin ser interferida por otras transacciones.

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2	
`READ`(b_1, x)		1000	1000	
`WRITE`$(b_1, x-100)$		900		
`READ`(b_2, x)				
	`READ`(b_1, y)			
	`WRITE`$(b_1, y-100)$	800		
	`READ`(b_2, y)			
	`WRITE`$(b_2, y+100)$		1100	
`WRITE`$(b_2, x+100)$		800	1100	✕

Una transacción no puede interferir con otra transacción

(pero queremos paralelizar lo más posible)

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Consistency

Al terminar una transacción los datos deben ser
correctos y consistentes según las restricciones de integridad.

Isolation

Cada transacción se ejecuta sin ser interferida por otras transacciones.

Durability

Si un transacción hace commit,
sus cambios sobrevivirán cualquier tipo de falla.

Garantías ACID

Durability

Si un transacción hace commit,
sus cambios sobrevivirán cualquier tipo de falla.

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2
`READ`(b_1, x)		1000	1000
`WRITE`$(b_1, x-100)$		900	
`READ`(b_2, x)			
`WRITE`$(b_2, x+100)$			1000
	`READ`(b_1, y)		
	`WRITE`$(b_1, y-100)$	800	
	`READ`(b_2, y)		
	`WRITE`$(b_2, y+100)$	800	1200
`ERROR`	`ERROR`	(datos en buffer)	



Una vez que haya un `COMMIT` exitoso, se persisten los cambios

¿cuándo NO tenemos ACID?

Atomicity

✗ Una transacción se ejecuta solo a medias
pero afecta el estado de la base de datos.

Consistency

✗ Al ejecutar la transacción,
la base de datos no satisface las restricciones de integridad.

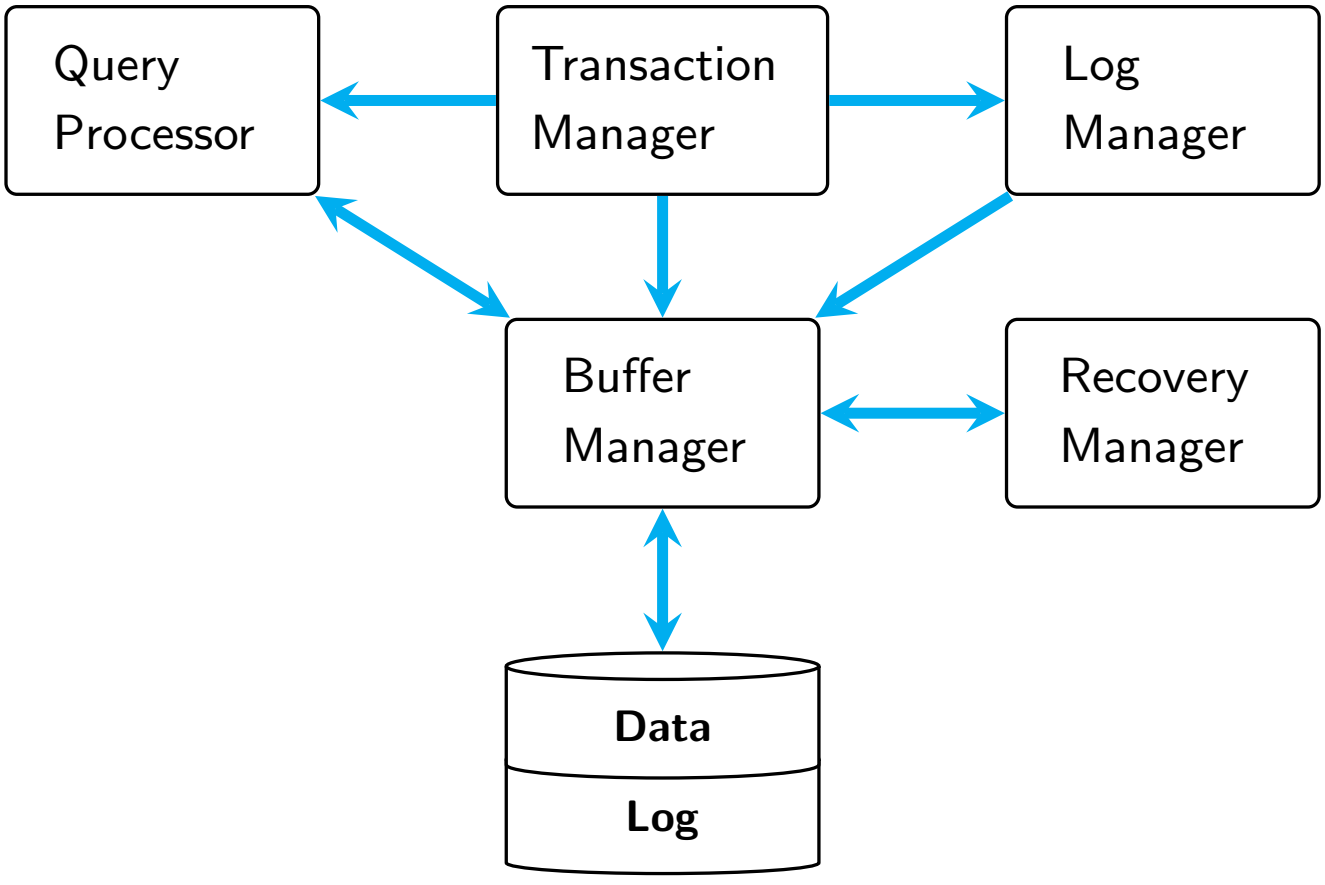
Isolation

✗ El resultado final de dos transacciones no es
equivalente a ejecutar cada transacción en serie.

Durability

✗ La base de datos se actualiza momentáneamente y
luego vuelve al estado anterior.

¿cómo podemos asegurar las propiedades ACID?



Próximas clases veremos el funcionamiento interno de las transacciones